

Программирование. Язык C

Лабораторная работа №4

Введение в функции. Базовая работа со строками
(однобайтовыми)

Редакция C11/C17

Общие требования

- Нормативный минимум: стандарт языка C11 или C17.
- Базовые заголовки для ЛР: `stdio.h`, `stdlib.h`, `string.h`, `ctype.h`.
- Для ввода строк используйте `fgets(...)` как основной кроссплатформенный вариант.
- Для преобразования строк в числа используйте `strtol(...)` (целые) и `strtod(...)` (вещественные) с проверкой ошибок (`errno`, `endptr`).
- Тип `size_t` — это платформо-зависимый беззнаковый целочисленный тип (не фиксированный `unsigned int`).
- Для единообразия вывода рекомендуется использовать латиницу.

Примечание по MSVC (опционально): функции `scanf_s` и `gets_s` допустимы как частный случай для Microsoft Visual C++, но не являются базовым переносимым вариантом для этой ЛР.

Примечание по locale.

- Базовый переносимый вариант: `setlocale(LC_ALL, "")`. Пустая строка `""` означает: библиотека C возьмёт параметры locale из переменных окружения операционной системы (обычно `LC_ALL`, `LC_CTYPE`, `LANG`).
- Опциональный частный пример для некоторых окружений:

```
setlocale(LC_ALL, "en_US.iso88591");
```

- Конкретная строка locale является платформо- и дистрибутивно-зависимой. На части систем она не установлена. Для выполнения данной лабораторной работы вызов `setlocale(...)` не является обязательным.

Комплект 1. Введение в функции

1.1. Факториал: цикл и рекурсия

- **Цель:** реализовать и сравнить два способа вычисления факториала: итеративный и рекурсивный.
- **Что подать на вход:** целое число $n \geq 0$.
- **Что считается корректным результатом:**
 1. есть две отдельные функции (через цикл и через рекурсию);
 2. обе функции для одного и того же n дают одинаковый ответ;

3. корректно обработан случай $n = 0$ (факториал равен 1).

- **Пояснение:** для учебной проверки удобно запускать программу на нескольких значениях n подряд (например, 0, 1, 5, 7) и сравнивать результаты двух реализаций.
- **Мини-пример:** вход 5 $\rightarrow$ обе функции печатают 120.

1.2. Обмен чётных/нечётных ячеек массива

- **Цель:** отработать передачу динамического массива в функцию и изменение данных “по месту”.
- **Что подать на вход:** динамический массив `int` из 12 элементов.
- **Что считается корректным результатом:** функция делает попарный обмен соседних элементов: индексы $0 \leftrightarrow 1, 2 \leftrightarrow 3, 4 \leftrightarrow 5, \dots, 10 \leftrightarrow 11$. При этом размер массива не меняется, а перестановка выполняется только внутри каждой пары.
- **Пояснение:** здесь “чётные и нечётные ячейки” означают **индексы** элементов, а не чётность самих значений.
- **Мини-пример:** $[1, 2, 3, 4, 5, 6] \rightarrow [2, 1, 4, 3, 6, 5]$.

1.3. Набор функций для матрицы `double`

- **Цель:** выделять, заполнять, печатать и освобождать двумерный динамический массив без утечек памяти.
- **Что подать на вход:** размеры матрицы `rows`, `cols` и значения элементов.
- **Что считается корректным результатом:**
 1. реализованы функции выделения и освобождения памяти;
 2. реализованы функции заполнения и печати;
 3. утечек памяти нет (каждый успешный `malloc/calloc` имеет соответствующий `free`).
- **Пояснение:** если на этапе выделения памяти под одну из строк возникает ошибка, необходимо освободить уже выделенные строки и вернуть признак ошибки, а не продолжать работу.
- **Мини-пример:** для матрицы 2×3 ввод 1 2 3 4 5 6 печатается как две строки 1 2 3 и 4 5 6.

1.4. Смешанное произведение трёх векторов в 3D

- **Цель:** вычислять смешанное произведение через разбиение задачи на небольшие понятные функции.
- **Что подать на вход:** три вектора длины 3 в декартовых координатах (x, y, z).
- **Обязательные интерфейсы функций:**

```
void cross3(const double a[3], const double b[3], double out[3]);
double dot3(const double a[3], const double b[3]);
double triple3(const double a[3], const double b[3], const double c[3]);
```

- **Что считается корректным результатом:**

1. реализована функция векторного произведения двух 3D-векторов (результат — вектор длины 3);
2. реализована функция скалярного произведения двух 3D-векторов (результат — число);
3. реализована функция смешанного произведения

$$[\mathbf{a}, \mathbf{b}, \mathbf{c}] = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c});$$

4. функция смешанного произведения использует две предыдущие функции, а не дублирует формулы вручную.

- **Пошаговый алгоритм:**

1. сначала вычислите $\mathbf{tmp} = \mathbf{b} \times \mathbf{c}$ с помощью `cross3`;
2. затем вычислите число $\mathbf{a} \cdot \mathbf{tmp}$ с помощью `dot3`;
3. это число и есть значение `triple3(a, b, c)`.

- **Справочная формула для векторного произведения:**

$$\mathbf{b} \times \mathbf{c} = (b_y c_z - b_z c_y, b_z c_x - b_x c_z, b_x c_y - b_y c_x).$$

- **Пояснение для интерпретации результата:** смешанное произведение — это ориентированный объём параллелепипеда, построенного на векторах $\mathbf{a}, \mathbf{b}, \mathbf{c}$. Если значение равно нулю, то векторы компланарны.
- **Мини-пример:** $\mathbf{a}=(1, 0, 0), \mathbf{b}=(0, 1, 0), \mathbf{c}=(0, 0, 1) \rightarrow [\mathbf{a}, \mathbf{b}, \mathbf{c}]=1$.

Комплект 2. Базовые операции со строками

2.1. Базовые строковые операции

- **Цель:** освоить базовые операции с C-строкой в максимально пошаговом режиме.
- **Что подать на вход:** строка длиной около 10 латинских символов в массив `char my_string[MY_SIZE]`. Размер `MY_SIZE` задайте через `#define` с запасом (например, если ожидаете 10 символов, удобно взять 32).
- **Пояснение терминов:**
 - **NUL** — служебный символ конца строки (байт `0x00`, в коде запись `'\\0'`).
 - **Буфер** — массив `char`, в котором хранится строка.
 - **Длина строки** — число видимых символов до NUL.
- **Что выполнить (шаги):**

1. Шаг 1. Ручной подсчёт длины по индексам.

- Что сделать: создайте счётчик `len1 = 0`; пройдите строку циклом `for` от индекса `0` до `NUL`; на каждом шаге увеличивайте счётчик.
- Что вывести: значение `len1`.
- Как проверить: длина должна совпасть с количеством введённых символов (без учёта `NUL`).

2. Шаг 2. Ручной подсчёт длины по указателю.

- Что сделать: создайте `char *p = my_string` и `len2 = 0`; пока условие `*p != '\0'` истинно, выполняйте `++len2; ++p;`.
- Что вывести: значение `len2`.
- Как проверить: значения `len1` и `len2` должны совпасть.

3. Шаг 3. Подсчёт длины библиотечной функцией.

- Что сделать: вычислите `size_t len3 = strlen(my_string)`.
- Что вывести: `len3` форматом `%zu`.
- Как проверить: `len1 == len2 == len3`.

4. Шаг 4. Копирование строки во второй буфер.

- Что сделать: создайте второй буфер `copy` достаточного размера; скопируйте строку из `my_string` в `copy`.
- Что вывести: исходную строку и копию.
- Как проверить: строки должны быть одинаковыми посимвольно.

5. Шаг 5. Конкатенация двух строк.

- Что сделать: задайте две строки в коде (например, "Hello" и "World"), подготовьте буфер назначения с запасом, выполните объединение.
- Что вывести: результат объединения.
- Как проверить: буфер назначения должен вмещать обе строки и завершающий `NUL`.

6. Шаг 6. Сравнение строк.

- Что сделать: сравните две строки через `strcmp` (или `strncmp`).
- Что вывести: числовой результат и словесную интерпретацию.
- Как проверить: `<0` — первая строка лексикографически меньше; `=0` — строки равны; `>0` — первая строка больше.

7. Шаг 7. Перевод регистра.

- Что сделать: для строки с буквами разного регистра получите версию в нижнем и верхнем регистрах через `tolower/toupper`.
- Что вывести: исходную строку, строку в нижнем регистре и строку в верхнем регистре.
- Как проверить: буквы корректно изменили регистр; не-буквенные символы не искажены.
- Важно: перед передачей символа в `tolower/toupper` делайте приведение к `unsigned char`.

- **Типичные ошибки и как их избежать:**
 - Ошибка: буфер слишком маленький. Решение: всегда закладывайте место под завершающий NUL.
 - Ошибка: неверная трактовка `strcmp`. Решение: проверяйте знак результата, а не конкретное число.
 - Ошибка: забыли ограничение длины при копировании/конкатенации. Решение: заранее вычисляйте, влезают ли данные.
 - Ошибка: передаётся отрицательный `char` в `tolower/toupper`. Решение: приводите к `unsigned char`.
- **Мини-пример:** вход `AbC12` → длина 5, нижний регистр `abc12`, верхний регистр `ABC12`.

2.2. Конвертация строк в числа

- **Цель:** безопасно преобразовывать текст в `int` и `double`, чтобы программа корректно реагировала на ошибочный ввод.
- **Что подать на вход:** две строки: одна с целым числом, другая с числом с плавающей точкой.
- **Что выполнить:**
 1. для целого числа используйте `strtol`;
 2. для вещественного числа используйте `strtod`;
 3. перед преобразованием обнулите `errno`;
 4. после преобразования проверьте:
 - был ли распознан хотя бы один числовой символ;
 - не произошло ли переполнение диапазона (`errno == ERANGE`);
 - не остались ли лишние символы после числа.
- **Что вывести:** результат преобразования и понятное сообщение об успешности или причине ошибки.
- **Мини-пример:** `"-42"` → `int -42`, `"3.14"` → `double 3.14`.

2.3. Классификация символов

- **Цель:** научиться классифицировать каждый символ строки с помощью функций из `ctype.h`.
- **Что подать на вход:** строка длиной 10–20 символов (цифры, латиница, пробелы, знаки пунктуации).
- **Что выполнить:**
 1. организуйте цикл по всем символам строки до NUL;
 2. для каждого символа проверьте его свойства (например, `isdigit`, `isalpha`, `isspace`, `ispunct`);
 3. сформируйте для каждого символа отдельную строку отчёта в понятном виде.

- **Что вывести:** список, где для каждого символа указано, чем он является (буква, цифра, пробел, пунктуация и т.д.).
- **Мини-пример:** для "A7, " вывести, что 'A' — буква, '7' — цифра, ' , ' — пунктуация, ' ' — пробел.

Техническая подсказка: при вызове `is*/to*` приводите символ к `unsigned char`, чтобы избежать неопределённого поведения на отрицательных значениях `char`.

Теоретическое приложение к ЛР №4

А.1. Функции в С

Функция в языке С — это отдельный фрагмент программы, который выполняет одну понятную задачу. Такой подход упрощает чтение кода, тестирование и поиск ошибок. Обычно мы сначала объявляем функцию (прототип), а затем пишем её тело (определение).

Прототип нужен, чтобы компилятор заранее знал:

- имя функции;
- тип возвращаемого значения;
- количество и типы параметров.

```
int factorial_iter(int n);    // прототип
int factorial_iter(int n) {  // определение
    ...
}
```

Важно помнить, что в языке С аргументы передаются по значению: функция получает копию данных, а не саму исходную переменную.

А.1.1. Передача по значению и через указатели

Когда функция получает параметр “по значению”, она работает с копией. Поэтому изменение параметра внутри функции не меняет исходную переменную в `main`.

Если нужно изменить данные снаружи, передают адрес переменной (указатель):

```
void swap2(int *a, int *b) {
    int t = *a; *a = *b; *b = t;
}
```

В этом примере меняются значения по адресам `a` и `b`, поэтому результат виден в вызывающем коде.

А.1.2. Рекурсия: базовый случай и стек вызовов

Рекурсия — это ситуация, когда функция вызывает саму себя. Такой подход удобен для задач, которые естественно распадаются на более простые подзадачи.

Чтобы рекурсия работала корректно, обязательно нужны:

- **базовый случай** (условие остановки);
- **шаг рекурсии**, приближающий к базовому случаю.

Пример для факториала:

```
int factorial_rec(int n) {  
    if (n <= 1) return 1;        // базовый случай  
    return n * factorial_rec(n - 1);  
}
```

Типичные ошибки:

- забыли базовый случай;
- шаг рекурсии не приближает к остановке (например, n не уменьшается);
- слишком глубокая рекурсия приводит к переполнению стека.

Мини-пример (итерация и рекурсия)

// $n = 5$: обе функции должны вернуть 120

А.2. Динамическая память и массивы

Динамическая память нужна, когда размер данных нельзя удобно зафиксировать на этапе компиляции. Память запрашивается во время работы программы и должна быть освобождена вручную.

Основные функции:

- `malloc` — выделяет блок памяти без обнуления;
- `calloc` — выделяет и заполняет нулями;
- `free` — освобождает блок.

Практические правила:

1. проверять результат выделения на `NULL`;
2. каждому успешному выделению должен соответствовать `free`;
3. после `free` полезно присвоить указателю `NULL` для защиты от повторного освобождения.

Паттерн попарного обмена для задания 1.2:

```
for (size_t i = 0; i + 1 < n; i += 2) {  
    int t = a[i];  
    a[i] = a[i + 1];  
    a[i + 1] = t;  
}
```

Мини-пример выделения и освобождения

```
int *a = malloc(12 * sizeof(*a));
if (!a) { /* обработка ошибки */ }
...
free(a);
a = NULL;
```

Типичная ошибка первокурсников — выделить память и забыть её освободить. В коротких программах это не всегда заметно, но в больших проектах такая ошибка приводит к утечкам памяти.

А.3. Двумерные динамические массивы

Один из самых понятных способов хранить динамическую матрицу в С — массив указателей на строки:

1. выделяется массив указателей `double **m`;
2. для каждой строки выделяется `m[i]`.

Освобождать такую структуру нужно в обратном порядке:

1. сначала все строки `free(m[i])`;
2. затем массив указателей `free(m)`.

Если на шаге выделения очередной строки возникла ошибка, необходимо освободить уже успешно выделенные строки и только потом завершить функцию с ошибкой.

Мини-пример

```
double **m = malloc(rows * sizeof(*m));
for (size_t i = 0; i < rows; ++i) {
    m[i] = malloc(cols * sizeof(*m[i]));
    if (!m[i]) { /* частичная очистка */ }
}
```

А.4. Смешанное произведение в 3D

Смешанное произведение трёх векторов $\mathbf{a} = (a_x, a_y, a_z)$, $\mathbf{b} = (b_x, b_y, b_z)$, $\mathbf{c} = (c_x, c_y, c_z)$ вычисляется как:

$$[\mathbf{a}, \mathbf{b}, \mathbf{c}] = \mathbf{a} \cdot (\mathbf{b} \times \mathbf{c}).$$

Полезная интерпретация: модуль этого значения равен объёму параллелепипеда, построенного на трёх векторах.

Свойства:

- результат — число (скаляр), а не вектор;
- модуль этого числа равен ориентированному объёму параллелепипеда;
- если три вектора компланарны, смешанное произведение равно нулю;
- знак зависит от ориентации тройки векторов.

Проверочный пример и разбиение на функции

```
// 1) cross3(b, c, tmp) -> вектор
// 2) dot3(a, tmp)      -> смешанное произведение
a = (1, 0, 0), b = (0, 1, 0), c = (0, 0, 1) => [a,b,c] = 1
```

Для лабораторной работы удобно придерживаться этой последовательности, потому что она прямо соответствует требуемому разбиению на функции.

А.5. Строки в С (однобайтовые)

С-строка — это массив `char`, который заканчивается служебным нулевым символом NUL (`0x00`, запись в коде `'\\0'`). Функции из `string.h` читают строку до первого NUL. Именно поэтому размер буфера и корректный NUL в конце строки критичны.

Для безопасного ввода рекомендуется `fgets`, потому что эта функция ограничивает число считываемых символов:

```
char s[64];
if (fgets(s, sizeof s, stdin)) {
    // s содержит не более 63 символов + '\\0' (NUL)
}
```

После `fgets` в буфере часто остаётся символ новой строки `'\n'`. Его можно удалить, если по логике задачи он не нужен:

```
size_t len = strlen(s);
if (len > 0 && s[len - 1] == '\\n') s[len - 1] = '\\0'; // NUL
```

А.6. Базовые операции со строками

В этом пункте собраны операции, которые используются чаще всего:

- `strlen` — длина строки без завершающего NUL;
- копирование: `strcpy`/ограниченные варианты;
- конкатенация: `strcat`/ограниченные варианты;
- сравнение: `strcmp`/`strncmp`.

Основной риск при работе со строками — переполнение буфера. Поэтому перед копированием и конкатенацией всегда проверяйте, поместятся ли данные в целевой массив вместе с завершающим NUL (`'\\0'`).

Мини-пример безопасной конкатенации

```
char dst[16] = "ab";
const char *src = "cd";
if (strlen(dst) + strlen(src) + 1 < sizeof(dst)) {
    strcat(dst, src);
}
```

A.7. Символьные функции из ctype.h

Функции семейства `is*` (например, `isdigit`, `isalpha`, `isspace`, `ispunct`) позволяют определить тип символа. Функции `tolower/toupper` переводят символ в нижний или верхний регистр.

Важно: аргумент таких функций нужно приводить к `unsigned char` (или использовать `E0F`), иначе для отрицательных значений `char` возможно неопределённое поведение.

Мини-пример

```
unsigned char c = (unsigned char)s[i];
if (isdigit(c)) { ... }
```

A.8. Преобразование строк в числа

Функции `atoi/atof` выглядят простыми, но почти не помогают диагностировать ошибки ввода. Для учебной и практической работы надёжнее использовать `strtol/strtod`:

- позволяют проверить, были ли распознаны цифры;
- позволяют определить хвостовые нечисловые символы;
- совместно с `errno` позволяют отслеживать переполнение.

Мини-пример проверки

```
errno = 0;
char *end = NULL;
long v = strtol(text, &end, 10);
if (end == text) { /* нет цифр */ }
if (errno == ERANGE) { /* выход за диапазон */ }
if (*end != '\0' && *end != '\n') { /* лишний хвост */ }
```

A.9. Locale и кодировки

Функция `setlocale` влияет на локализованное поведение стандартной библиотеки C: на правила обработки символов, формат некоторых данных и поведение ряда функций.

- `setlocale(LC_ALL, "")` — базовый переносимый вариант: пустая строка означает “взять locale из окружения” (обычно переменные `LC_ALL`, `LC_CTYPE`, `LANG`).
- `setlocale(LC_ALL, "en_US.iso88591")` — опциональный частный пример, который работает только если такая locale установлена в ОС.

Для данной лабораторной работы вызов `setlocale` не обязателен. Если ввод и вывод выполняются в латинице, задания можно корректно выполнить без дополнительной настройки locale.